RAG Assignment: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Roll No: 2023111002

Code: `https://github.com/DheeruYS/RAG-LMA-A1` (file `rag_rerank_gdpr.ipynb`)

1 Setup

Document. I used the EU General Data Protection Regulation (GDPR), the official 88-page EUR-Lex PDF. I picked it because it is full of general rules followed by "shall not apply where..." exceptions, and because its 173 recitals repeat the wording of the 99 articles. That gives a lot of passages that look similar to each other, which is exactly where a weak retriever gets confused.

Chunking. I removed the page headers and the footnotes. The footnotes are printed in a smaller font, so I filtered them out by font size. I then split the text at recital/article/paragraph boundaries and packed consecutive paragraphs into chunks of at most 180 words, with a small overlap. This gave **430 chunks**, about 136 words each on average. Both pipelines use exactly this same set of chunks.

Models.

- Bi-encoder: `all-MiniLM-L6-v2` (384-dim). Vectors are normalised, so inner product = cosine similarity. Search is exact over all 430 vectors.
- Cross-encoder: `cross-encoder/ms-marco-MiniLM-L-6-v2`.
- LLM: `Qwen2.5-3B-Instruct`, run locally on an RTX 4070 laptop GPU. I used greedy decoding, so answers are deterministic and any difference between Answer 1 and Answer 2 comes only from the context.

Pipelines. Pipeline 1 takes the bi-encoder Top-10 and passes the first 3 chunks to the LLM. Pipeline 2 takes the *same* Top-10, scores all 10 (query, chunk) pairs with the cross-encoder, re-sorts them, and passes the new top 3 to the LLM. The prompt is identical in both cases: answer only from the passages, quote exact figures and exceptions, and say so if the answer is not there.

2 Architecture Comparison

Bi-encoder. The query and the chunk are encoded *separately*, and each one is turned into a single vector. Relevance is then the cosine similarity between the two vectors. Because chunks never need to see the query, all chunk vectors can be computed once, offline. At query time we only encode the query and do a fast vector search. The downside is that the entire chunk has to be compressed into one vector before the model knows what will be asked. Small but important details, like a "not", a specific amount, or which paragraph holds the exception, get averaged away. In practice it behaves like a "same topic" detector.

Cross-encoder. The query and the chunk are joined into one input (`[CLS] query [SEP] chunk`) and go through the transformer *together*. Self-attention lets every query word look at every chunk word in every layer, and the model outputs one relevance score. So it can check whether "not required" in the question actually matches "shall not be required" in the passage, instead of just noticing that both talk about data breaches. The price is that nothing can be pre-computed. Every (query, chunk) pair needs its own full forward pass.

3 Qualitative and Rank Shift Analysis

For each test query I marked the *critical chunk*, meaning the chunk that actually contains the answer, using a regex on the exact legal wording. This way the rank change is measured, not judged by eye. Table 1 shows the results.

Failure type	Query	Critical chunk	Baseline	Re-ranked
Keyword trap	How much can a company be fined for failing to report a data breach?	Art. 83(4)	#7	#3
Exception	When is a controller not required to tell affected individuals about a breach?	Art. 34(3)	#5	#2
Negation	Is it forbidden to process biometric data to identify a person?	Art. 9(1)	#8	#2
Role limit	Can a member of a supervisory authority have another paid job?	Art. 52(3)	#7	#1
Buried chunk	What is the maximum fine for violating the basic principles of processing?	Art. 83(5)	#8	#1
Counter-ex. 1	When does the right to be forgotten not apply?	Art. 17(3)	#4	#6
Counter-ex. 2	Can public authorities rely on legitimate interests as a legal basis?	Art. 6(1)	#90	not in top-10

Table 1: Rank of the critical chunk before and after re-ranking. Only ranks 1–3 reach the LLM.

Buried chunk (maximum fine). The bi-encoder ranked the general Art. 83 chunks and fine-related recitals above the paragraph that actually lists "the basic principles for processing". So the baseline LLM only saw the EUR 10M paragraph and answered *"10,000,000 EUR or 2%"*, which is **wrong**. After re-ranking, Art. 83(5) moved from #8 to #1 and Answer 2 was *"20,000,000 EUR or 4%"*, which is correct.

Exception (breach communication). The baseline top 3 were all recitals about breaches. The LLM picked up a sentence from Recital 85 and said the controller does not need to inform people if the breach is *"unlikely to result in a risk"*. That condition is actually for notifying the *supervisory authority* (Art. 33), not the individuals, so the rule was **misstated**. Re-ranking promoted Art. 34(3) from #5 to #2, and Answer 2 listed the three real exceptions: encryption or similar protection, later measures that remove the high risk, and disproportionate effort (in which case a public notice is needed instead).

Negation (biometric data). Recital 51 says biometric data *"should not be processed, unless. . . "*. Using it, the baseline LLM concluded *"it is not forbidden but requires specific authorization"*. This is the classic case where the negative and positive meaning get blurred. Art. 9(1) clearly says such processing *"shall be prohibited"*. After re-ranking (#8 → #2), Answer 2 said processing for identification is *"generally forbidden"*, which is the right conclusion. However, its description of the exceptions was muddled, so the improvement here is only partial.

Keyword trap (fine for not reporting a breach). The query shares the words "fine" and "breach" with the EUR 20M paragraph and with the breach recitals. The correct paragraph, Art. 83(4), never uses the word "breach". It only says "obligations . . . pursuant to Articles 8, 11, 25 to 39". The baseline gave a confident **hallucinated** answer: *"up to 4% . . . or 20 million euros"*. With re-ranking, the right chunk reached #3, but the 3B model did not make the extra step that breach reporting is Article 33, so it answered *"the provided context does not contain the answer"*. Not a perfect answer, but a refusal is much better than a wrong figure.

Role limit (paid job). The binding rule is worded as *"shall not . . . engage in any incompatible occupation, whether gainful or not"*, with no mention of "paid" or "job". Art. 52(3) moved from #7 to #1. Here both answers were correct, though, because Recital 121 repeats the same sentence and was already at #1 in the baseline. This shows that the recitals sometimes hide retrieval mistakes.

Where re-ranking did not help.

- *Right to be forgotten:* the cross-encoder pushed Art. 17(3) down from #4 to #6 and preferred the recitals that talk about the right in general. Both answers then listed a *ground* for erasure (data no longer needed) as if it were an exception. This is wrong in both pipelines.
- *Public authorities and legitimate interests:* the Art. 6(1) sentence was at rank #90, so no re-ranker of the Top-10 can bring it back. Re-ranking can only reorder what stage 1 already found. Recital 47 repeats the rule and was in both contexts. Answer 1 eventually mentioned the restriction, though it contradicted itself, while Answer 2 dropped it and said public authorities *can* rely on it. So even with a good chunk at #1, a small LLM can still ignore it.

Overall numbers. On 20 labelled questions (the 5 stress tests plus 15 normal ones), the critical chunk was in the LLM's context (Hit@3) for **80%** of questions with the baseline and **100%** after re-ranking. MRR@10 went from 0.713 to 0.867. Recall@10 is 100% for both by construction, since both pipelines use the same 10 candidates. On the normal questions both pipelines were mostly already at rank 1, so the gain comes almost entirely from the harder questions.

4 Latency Profiling

All timings are on an RTX 4070 laptop GPU, measured after a warm-up, and averaged over 200 runs (20 queries × 10 repetitions).

Pipeline	Stage	Mean (ms)	Median (ms)	p95 (ms)
1	Query embedding (bi-encoder)	3.11	2.89	4.48
1	Vector search over 430 chunks	0.06	0.06	0.08
1	**Total baseline retrieval**	**3.17**	2.94	4.55
2	Cross-encoder on 10 pairs	13.48	13.39	15.15
2	**Total retrieval + re-rank**	**16.66**	16.38	19.57
both	LLM answer generation	1520 / 2113		

Table 2: Latency per query. The LLM row shows Pipeline 1 / Pipeline 2.

Re-ranking makes retrieval about **5× slower** (+13.5 ms), but this is tiny compared to the LLM, which takes 1.5–2.1 s. The LLM difference between the pipelines is not caused by re-ranking. Answer 2 was simply longer on average (67 vs. 47 tokens), and the speed per token was the same in both cases (about 33 ms/token). So in end-to-end terms, re-ranking adds less than 1% to the total time.

5 Scaling Question

Why not just run the cross-encoder over the whole collection from the start? Because its cost grows with the number of chunks *for every single query*, and none of it can be prepared in advance. I measured this directly:

- Bi-encoder over all 430 chunks: **3.2 ms** per query (the chunk vectors were already computed offline, in 0.6 s total).
- Cross-encoder on the 10 candidates: **13 ms** per query.
- Cross-encoder on all 430 chunks: **476 ms** per query, i.e. about 1.1 ms per pair.

That is already 150× slower than the bi-encoder for just one document. For a collection of 1 million chunks it would be around 18 minutes per query on this GPU. The bi-encoder, in contrast, only encodes the query once, and with an approximate nearest-neighbour index (e.g. FAISS/HNSW) the search time grows much more slowly than the collection size. Memory is also an issue: a bi-encoder

stores one small vector per chunk, while a cross-encoder has nothing to store and must re-read all the text each time.

Also, running the cross-encoder on everything does not fix every case. In my test it put the right-to-be-forgotten exception at rank 20 out of 430. So the sensible design is the two-stage one: a cheap, high-recall bi-encoder narrows the corpus down to a few candidates, and the expensive but precise cross-encoder only reorders those. In my experiments this gave the full Hit@3 gain for about 13 ms of extra time.

6 Takeaways

- Re-ranking fixed the cases where the answer was in the Top-10 but below rank 3. In 3 of the 5 stress tests it turned a wrong answer into a correct (or mostly correct) one, and in the keyword-trap case it turned a hallucination into a refusal.
- Re-ranking cannot fix poor first-stage recall. Retrieving more than 10 candidates, or a better bi-encoder, would help with cases like the Art. 6 query.
- The cross-encoder sometimes prefers general "topic" passages over the specific exception, and a small 3B LLM does not always use the right chunk even when it is given one. Better retrieval helps a lot, but it is not the whole story.